1. Overview

ComPDF Conversion SDK is a high-performance library designed for extracting and transforming the data within your PDF files, such as text, images, tables, links, and annotations, into various file formats. Our Conversion SDK retains the original document layout and the properties of the file data, ensuring a seamless document conversion experience.

Effortlessly integrate the ComPDF Conversion SDK into your projects in just a few steps, and enable the following file format conversions:

- Convert PDF to Word (.docx)
- Convert PDF to Excel (.xlsx)
- Convert PDF to PowerPoint (.pptx)
- Convert PDF to HTML (.html)
- Convert PDF to CSV (.csv)
- Convert PDF to Image (.png, .jpg, .jpeg,.jpeg2000, .bmp, .tiff, .tga, .gif, .webp)
- Convert PDF to Plain Text (.txt)
- Convert PDF to Rich Text Format (.rtf)
- Convert PDF to Searchable PDF (.pdf)
- Convert PDF to OFD (.ofd)
- Convert PDF to Structured Data (.json)
- Convert PDF to Markdown (.md)

Harness the power of ComPDF Conversion SDK and take your document processing to the next level with intelligent tools designed for accuracy and efficiency. To enhance your format conversion results, ComPDF offers AI-powered document tools with the following capabilities:

- Optical Character Recognition (OCR)
- Layout Analysis
- Table Recognition

1.1 Why ComPDF Conversion SDK

- Mature Technology

  With years of technology accumulation, we have established a complete mechanism of product iteration to offer a continuous guarantee for product competitiveness.

- Complete PDF and Format Conversion Functionalities

  Our comprehensive features can meet diverse needs and are easy for our customers to use without training costs.

- High-quality Service

  professional service and technical support to quickly respond to users' feedback through onsite service or remote support like telephone, email, etc.

- Independent Intellectual

Property Rights
Our technology is independent and compliant with ISO, helping enterprises conduct international business without considering copyright risks.

1.2 ComPDF Conversion SDK

The ComPDF Conversion SDK is designed to facilitate the effortless conversion of PDF files into various other formats, all while preserving the original layout and formatting of the documents. In this guide, we will explore the ComPDF Conversion SDK and demonstrate how to utilize it within your C++ projects.

1.3 License & Trial

The ComPDF Conversion SDK is a commercial SDK that requires a license to grant developers the right to develop and distribute their applications. In development mode, each license is only valid for one device ID. ComPDF provides flexible licensing models, please contact our marketing team for more information. However, even if you have a license, it is prohibited to distribute any documents, sample code, or source code of the ComPDF Conversion SDK to any third parties.

If you do not have a License, please feel free to contact the ComPDF Team to obtain a License to try the ComPDF Conversion SDK.

2. Get Started

2.1 Requirements

Before starting, please make sure that you have already met the following prerequisites.

2.1.1 Get ComPDF License Key

ComPDF provides two types of license key: 30-day free trial license, and commercial license.

How to Get Free Trial License

Contact our sales team and we'll send you a 30-day free trial license for ComPDF Conversion SDK.

How to Get Commercial License

ComPDF Conversion SDK is a commercial SDK that requires a license for application release. Any documents, sample code, or source code distribution from the released package of ComPDF to any third party is prohibited.

Contact Sales

To get commercial license for ComPDF Conversion SDK, feel free to contact our sales team.

For C++ Conversion SDK, commercial license must be bound to your developer device ID (How to find the developer device ID), and each license is only valid for one device ID in development mode.

2.1.2 Download Conversion SDK

Contact usto obtain the ComPDF C++ Conversion SDK.

2.1.3 System Requirements

Development Platform	System Requirements	Development Environment	Notice
Windows	- Windows 7, 8, 10, and 11 (32-bit, 64-bit) - Need to support C++ Version 11 or higher.	Visual Studio 2022 or higher.	Samples have been tested on Windows 10 and 11.
Linux	- Linux x64. - Need to support C++ Version 11 or higher.	/	Samples have been tested on Ubuntu 20.04.
Mac	- Mac OS 10.14 or higher (Intel, Apple Silicon). - Need to support C++ Version 11 or higher.	Xcode 13.0 or higher.	Samples have been tested on Mac Intel and Mac Apple Silicon.

2.2 SDK Package Structure

You can contact us to get our PDF format conversion SDK package. The ComPDF Conversion C++ SDK contains the following files:

- ***"doc"*** - API reference and developer guide.
- ***"include"*** - header files for ComPDF Conversion SDK API.
- ***"lib"*** - Include the ComPDF dynamic library (x86 and x64).
- ***"resource"*** - Include DocumentAI model resources.
- ***"samples"*** - A folder containing sample projects.
- ***"legal.txt"*** - Legal and copyright information.
- ***"release_notes.txt"*** - Release information.

2.3 Apply the License Key

If you don't have a license key, please check out how to obtain a license key.

ComPDF Conversion SDK currently supports offline authentication to verify license keys.

Learn about:

What is the authentication mechanism of ComPDF's license?

2.3.1 Copy the License Key

Accurately obtaining the license key is crucial for the application of the license.

1. In the email you received, locate the XML file containing the license key.
2. Open the XML file, and determine the license type based on the field. If online is present, it indicates an online license. If offline is present or if the field is absent, it indicates an offline license.

Online License:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<license version="1">
    <platform>windows</platform>
    <starttime>xxxxxxxx</starttime>
    <endtime>xxxxxxxx</endtime>
    <type>online</type>
    <key>LICENSE_KEY</key>
</license>
```

Offline License:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<license version="1">
    <platform>windows</platform>
    <starttime>xxxxxxxx</starttime>
    <endtime>xxxxxxxx</endtime>
    <key>LICENSE_KEY</key>
</license>
```

3. Copy the value located at the LICENSE_KEY position within the LICENSE_KEY field. This is your license key.

2.3.2 Apply the License Key

You can perform offline authentication using the following method:

```
// Include the header file.
#include "compdf_conversion.h"

// Include namespaces.
using namespace compdf;
using namespace compdf::base;
using namespace compdf::common;
using namespace compdf::conversion;

// Verify the license.
const char* license = " ";
ErrorCode code = LibraryManager::LicenseVerify(license, "device_id", "app_id");
```

```
if (code != ErrorCode::Success) {
    return false;
}

// Initialize conversion resources.
const char* resource_path = " ";
LibraryManager::Initialize(resource_path);
```

2.4 How to Run a Demo

2.4.1 Windows

ComPDF Conversion SDK provides a demo in the **"samples"** folder. You can follow the following steps to run the demo.

1. Load the Visual Studio solution file **"demo_vs2022.sln"** in the **"\samples"** folder.
2. Build the demo by clicking **Build > Build Solution**. After the build is completed, the executable file **".exe"** will be generated in the **"\samples\bin"** folder. The name of the executable file depends on the build configuration.
3. Double-click the executable file to run the demo.

Output files (Word, Excel, PowerPoint, etc.) will be generated in the **"samples/output_files"** folder.

2.4.2 Linux and Mac

ComPDF Conversion SDK provides demos in the **"samples"** folder. Before running the demo, make sure you have configured your environment correctly and installed CMake (3.0 or higher) on your machine. To run the demo, you can follow these steps:

1. Open a terminal window and navigate to the **"samples"** folder of the ComPDF Conversion SDK for Linux.
2. Enter the following command to run the demo.

```
./RunDemo.sh
```

Output files (Word, Excel, PowerPoint, etc.) will be generated in the **"samples/output_files"** folder.

3. Conversion Guides

ComPDF Conversion SDK allows developers to use a simple API to convert PDF to commonly used file formats such as Word, Excel, PowerPoint, HTML, CSV, PNG, JPEG, RTF, TXT, Searchable PDF, OFD, JSON, and Markdown. It provides a wide range of customized conversion options, such as whether to include images or annotations in PDF documents, whether to enable OCR, whether to enable layout analysis, and more.

3.1 Initialize Library Resources

Overview

Initialize the necessary file and memory resources required by the ComPDF Conversion SDK.

Notes

- You must initialize SDK resources before calling any conversion interface.
- When using OCR, Layout Analysis, Table Recognition, PDF to Searchable PDF, or PDF to OFD, make sure the DocumentAI model resources in the `resource` directory are available.

Example

```
// Initialize SDK resources.
LibraryManager::Initialize("ComPDF_Conversion_SDK/resource");
```

3.2 Set DocumentAI Model

Overview

Before using OCR, Layout Analysis, Table Recognition, PDF to Searchable PDF, or PDF to OFD, you need to set the DocumentAI model path first.

`SetDocumentAIModel` supports an optional `gpu_id` parameter used to specify the GPU device index for the AI model. When `gpu_id` is `-1`, GPU acceleration is disabled.

Example

```
LibraryManager::SetDocumentAIModel("path/documentai.model", -1);
```

Set AI Model Instance Count

If you need to control the number of Layout Analysis and Table Recognition model instances, call the following interface:

```
LibraryManager::SetDocumentAIModelCount(1, 1);
```

The first parameter indicates the number of Layout Analysis model instances, and the second parameter indicates the number of Table Recognition model instances.

Use Your Own AI Engine (SDK v4.1.0+)

This option is available only in SDK v4.1.0 or later. If you prefer to run OCR, Layout Analysis, or Table Recognition with your own model or a third-party service instead of the bundled DocumentAI model, the SDK exposes callback hooks on `CConvertCallback` that let you supply the results as JSON. See 3.11 Use Custom AI Models via Callbacks for details. When all three capabilities you need are covered by your own callbacks, `SetDocumentAIModel` does not have to be called.

3.3 Get Conversion Progress

ComPDF Conversion SDK obtains conversion progress through the callback functions in `CConvertCallback`. The following example demonstrates how to get the conversion progress while performing a PDF to Word task:

```
void Progress(int current_page_count, int total_page_count)
{
  std::cout << "progress: " << current_page_count << " / " << total_page_count << std::endl;
}

ConvertOptions opt;
CConvertCallback callback = {};
callback.handle = nullptr;
callback.progress = Progress;
callback.cancel = nullptr;

CPDFConversion::StartPDFToWord("input.pdf", "password", "path/output.docx", opt, &callback);
```

The `handle` field is maintained internally by the SDK during conversion, so it should be initialized to `nullptr` when passed in.

3.4 Cancel Conversion Task

ComPDF Conversion SDK supports interrupting your ongoing conversion task at any time through the `cancel` callback in `CConvertCallback`. When the `cancel` callback returns `true`, the current conversion task stops as soon as possible.

```
bool Cancel()
{
  return false;
}

ConvertOptions opt;
CConvertCallback callback = {};
callback.handle = nullptr;
callback.progress = nullptr;
callback.cancel = Cancel;

CPDFConversion::StartPDFToWord("input.pdf", "password", "path/output.docx", opt, &callback);
```

If you need to cancel the conversion at a specific time, you can return `true` from `Cancel` based on external state.

3.5 Select Page Range for Conversion

ComPDF Conversion SDK supports converting a specified page range. When an empty string is passed, all pages will be converted. If the page range exceeds one page, you can also choose to enable the `output_document_per_page` option to output each PDF page as a separate file. The following example demonstrates how to specify a page range when performing a conversion task:

```
ConvertOptions opt;
opt.output_document_per_page = true;
opt.page_ranges = "1-3,5,7-9";
```

3.6 Contain Image and Annotation Options

Overview

In the process of converting PDF documents into various formats, ComPDF Conversion SDK offers two additional options for users: one option to determine whether images are included in the generated document, and another to decide if annotations from the PDF file are to be retained.

- With the "Include Images" option enabled, ComPDF Conversion SDK will extract the images from the PDF document and embed them in the corresponding pages and positions in the output file. For areas with overlapping images, ComPDF Conversion SDK merges these images into one and embeds it into the exact location on the corresponding page of the output file.
- When the "Include Annotations" option is selected, most annotations are converted into raster images and embedded at the respective positions within your document. However, certain types of annotations, such as highlights, underlines, strikeouts, and squiggly, are converted into their respective formatting equivalents in the converted Word, PPT, and HTML documents, and are marked over the corresponding text. It is important to note that the conversion won't be 100% accurate in every instance.

In the ComPDF Conversion SDK, the options of including image and annotation are commonly used in the following format conversion:

- PDF to Word
- PDF to Excel
- PDF to PowerPoint
- PDF to HTML
- PDF to RTF
- Extract PDF to JSON
- Extract PDF to Markdown

About Text Markup Annotation

- **Highlight:** When converting PDF to Word format keeping the highlight markups, it is important to note that Microsoft Word only supports 15 highlighter colors. To approximate the original document's appearance as closely as possible, text in the Word document will be marked with a text background color that matches the color of the original document's highlight annotation. For conversions to Microsoft PPT format, the native highlighting feature within the format is used to mark the text. In the case of converting to HTML format, a unique `<span>` tag is created for the marked text, and the background style is set to match the color of the corresponding annotation in the original document.
- **Underlines & Squiggly:** When converting PDF to Word or PPT formats keeping the underline and squiggly markups, the marked text will be marked with the same style in Microsoft Office. When converted to HTML format, the marked text will be styled to display the same effect. However, if a paragraph of text in the original document is marked by both underline and squiggly, then the text will only be marked with one type (Because squiggly is actually a type of underline in Word, PPT, and HTML formats).
- **Strikeout:** When converting strikeout markups to Word and PowerPoint formats, the marked text will be added with a strikeout natively supported by Microsoft Office. However, in these two file formats, the color of the strikeout itself cannot be the same as that in the original PDF document because the

strikeout color in Word and PPT will only change according to the color of the marked text font itself. When converted to HTML format, the same strikeout color as the original document will be displayed.

Sample

This Sample demonstrates how to use the ComPDF Conversion SDK to convert a PDF document to a Word document with the selected options: Include images and annotations.

```
ConvertOptions opt;
// Setting contains image and contains Annotation.
opt.contain_image = true;
opt.contain_annotation = true;
CPDFConversion::StartPDFToWord("input.pdf", "password", "path/output.docx", opt);
```

3.7 Page Layout Mode

In certain formats, the page layout mode plays a key role in the quality of the converted document. ComPDF Conversion SDK supports two layout modes: Flow Layout and Box Layout.

- **Flow Layout:** This layout uses paragraph indentations, columns, and tab positions to adjust the content. Its main advantage is flexibility; content can flow automatically as the document is edited, and it adapts to various screen sizes on different devices. This layout also supports structured maintenance and can implement consistent global formatting through style templates (e.g., titles, body text). Common use cases include documents that are frequently modified, such as reports, manuals, and dynamic tables.
- **Box Layout:** Based on the PDF's "digital paper" model, this layout accurately positions every element (text, images, tables) on the page using a coordinate system (e.g., text is positioned 5 cm from the top and 3 cm from the left). The main advantage is high-precision rendering, which ensures consistency across different platforms. This layout is particularly useful for documents requiring precise reproduction, such as contracts, design drafts, and academic papers.

In the ComPDF Conversion SDK, page layout modes are commonly used in the following format conversions:

- PDF to Word
- PDF to HTML

Sample

This example demonstrates how to convert a PDF document to Word with Flow Layout and Box Layout:

```
ConvertOptions opt;
// Set Flow Layout mode.
opt.page_layout_mode = PageLayoutMode::e_Flow;
CPDFConversion::StartPDFToWord("input.pdf", "password", "path/output.docx", opt);

// Set Box Layout mode.
opt.page_layout_mode = PageLayoutMode::e_Box;
CPDFConversion::StartPDFToWord("input.pdf", "password", "path/output.docx", opt);
```

3.8 OCR

Overview

OCR (Optical Character Recognition) is the process of converting images of typed, handwritten, or printed text into machine-encoded text.

OCR is commonly used for text recognition and extraction from the following types of documents:

- Non-editable scanned PDF files.
- Photographs of documents.
- Scene photos such as advertising layouts, signboards, etc.
- Identification cards, passports, vehicle license plates, and other official plates.
- Invoices, bills, receipts, and other financial documents.

The following features support OCR:

- PDF to Word
- PDF to Excel
- PDF to PowerPoint (PPT)
- PDF to HTML
- PDF to Rich Text Format (RTF)
- PDF to Text (TXT)
- PDF to CSV
- PDF to Searchable PDF
- Extract PDF to JSON
- Extract PDF to Markdown

OCR Language Support of ComPDF Conversion SDK:

Script / Notice	Language (Native)	Language (In English)
Latn; American	English	English
Latn; Canadian	Français canadien	French
Hans/Hant	中文简体	Chinese (Simplified)
Hans/Hant	中文繁体	Chinese (Traditional)
Jpan	日本語	Japanese
Kore	한국어	Korean
Latn	Deutsch	German
Latn	Српски (латиница)	Serbian (latin)
Latn	Occitan, lenga d'òc, provençal	Occitan

Script / Notice	Language (Native)	Language (In English)
Latn	Dansk	Danish
Latn	Italiano	Italian
Latn; European	Español	Spanish
Latn; European	Português (Portugal)	Portuguese
Latn	Te reo Māori	Maori
Latn	Bahasa Melayu	Malay
Latn	Malti	Maltese
Latn	Nederlands	Dutch
Latn; Bokmål	Norsk	Norwegian
Latn	Polski	Polish
Latn	Română	Romanian
Latn	Slovenčina	Slovak
Latn	Slovenščina	Slovenian
Latn	shqip	Albanian
Latn	Svenska	Swedish
Latn	Swahili	Swahili
Latn	Wikang Tagalog	Tagalog
Latn	Türkçe	Turkish
Latn	oʻzbekcha	Uzbek
Latn	Tiếng Việt	Vietnamese
Latn	Afrikaans	Afrikaans
Latn	Azərbaycan	Azerbaijani
Latn	Bosanski	Bosnian
Latn	Čeština	Czech
Latn	Cymraeg	Welsh
Latn	Eesti keel	Estonian
Latn	Gaeilge	Irish
Latn	Hrvatski	Croatian

Script / Notice	Language (Native)	Language (In English)
Latn	Magyar	Hungarian
Latn	Bahasa Indonesia	Indonesian
Latn	Íslenska	Icelandic
Latn	Kurdî	Kurdish
Latn	Lietuvių	Lithuanian
Latn	Latviešu	Latvian

Set OCR Language

In the current mainline version, OCR languages should be passed through `ConvertOptions.languages` for each conversion task, rather than through a separate global interface.

```
ConvertOptions opt;
opt.enable_ocr = true;
opt.languages = {OCRLanguage::e_English};
CPDFConversion::StartPDFToWord("word.pdf", "password", "path/output.docx", opt);
```

OCR Options

Different OCR options can be selected according to actual needs. Below are the currently supported OCR options.

- **e_InvalidCharacter:** Recognizes invalid/garbled characters in the PDF document through OCR, while normal characters are not processed by OCR.
- **e_ScanPage:** Recognizes scanned pages in the PDF document through OCR, while editable pages are not processed by OCR.
- **e_InvalidCharacterAndScanPage:** Recognizes both invalid characters and scanned pages in the PDF document through OCR.
- **e_All:** Recognizes all pages and characters in the PDF document through OCR.

Preserve Page Background

When OCR is enabled, you can choose whether to enable the `contain_page_background_image` option. If this option is enabled, the original page background image of the PDF will be preserved. If it is disabled, the image result detected during page layout analysis will be retained.

Notice

- The quality of the OCR result depends on the quality of the input image. If the input image has a low resolution, the OCR result quality will be affected. A good rule of thumb is that the more pixels in the character shapes, the better. If the character bounding box is smaller than 20x20 pixels, OCR quality will drop exponentially. The ideal image is a grayscale image with a resolution around 300 DPI.
- When performing OCR, make sure the OCR language setting matches the language in the PDF document

to achieve the best OCR conversion quality.
- OCR functionality currently does not support operating systems lower than Windows 10.

Converting Images to Other Document Formats

The OCR function also supports converting input images into Word, Excel, PPT, HTML, CSV, RTF, TXT, JSON, and other formats. This sample demonstrates how to use the ComPDF OCR function to convert image files to a DOCX file.

```
// Set the DocumentAI model path.
LibraryManager::SetDocumentAIModel("path/documentai.model");

ConvertOptions opt;
// Enable OCR option.
opt.enable_ocr = true;
// Set the OCR language for this task.
opt.languages = {OCRLanguage::e_English};
// Supports jpg, jpeg, png, bmp, tiff, and webp formats.
CPDFConversion::StartPDFToWord("input.png", "", "path/output.docx", opt);
```

Sample

This Sample demonstrates how to use the ComPDF OCR function to convert a PDF to DOCX file.

```
// Set the DocumentAI model path.
LibraryManager::SetDocumentAIModel("path/documentai.model");

ConvertOptions opt;
// Enable OCR option.
opt.enable_ocr = true;
opt.languages = {OCRLanguage::e_English};
CPDFConversion::StartPDFToWord("word.pdf", "password", "path/output.docx", opt);
```

3.9 Layout Analysis

Overview

Layout analysis is the process of leveraging Artificial Intelligence (AI) technology to parse and understand the structure of a document's layout. Its primary goal is to extract text, images, tables, layers, and other data from the input documents.

Layout analysis has several common use cases, including:

- **Intelligent recognition of tables within PDF documents:** This feature is particularly useful for analyzing company financial statements, invoices, bank statements, experimental data, medical test reports, and more.
- **Smart extraction of text, images, or tables from PDF documents through layout analysis:** This functionality greatly aids in the analysis and extraction of information from identification cards, receipts, licenses, documents, ancient books, and other various types of files.

Features that support Layout Analysis:

- PDF to Word
- PDF to Excel
- PDF to PowerPoint (PPT)
- PDF to HTML
- PDF to RTF
- PDF to TXT
- PDF to CSV
- Extract PDF to JSON
- Extract PDF to Markdown

Notice

- You need to load the DocumentAI model before using layout analysis, **or** plug in your own AI engine via the callbacks described in 3.11 Use Custom AI Models via Callbacks.
- When the OCR is enabled, the layout analysis is automatically enabled.
- AI table recognition is a separate stage controlled by its own option. See 3.10 Table Recognition for details.

Sample

This Sample demonstrates how to use Layout Analysis to convert a PDF to a DOCX file.

```
// Set the DocumentAI model path.
LibraryManager::SetDocumentAIModel("path/documentai.model");

ConvertOptions opt;
// Enable layout analysis option.
opt.enable_ai_layout = true;
CPDFConversion::StartPDFToWord("word.pdf", "password", "path/output.docx", opt);
```

3.10 Table Recognition

Overview

Table Recognition reconstructs the internal structure of tables detected during layout analysis, including rows, columns, merged cells, and cell boundaries, so that the converted document preserves the original tabular semantics instead of producing a flat grid of text fragments.

It is controlled by the independent option `enable_ai_table_recognition`, which is **enabled by default**. The table model is only invoked for table regions reported by layout analysis whose detection confidence is below the trusted threshold; high-confidence native PDF tables bypass the model to save inference time.

Typical scenarios that benefit from Table Recognition:

- **Borderless or partially bordered tables**, where ruling lines alone cannot describe the structure.
- **Tables with merged header cells, multi-row headers, or spanning cells**, such as financial statements, lab reports, and invoices.

- **Scanned tables processed by OCR**, where geometric reconstruction is required before cell-level data extraction.

Features that support Table Recognition:

- PDF to Word
- PDF to Excel
- PDF to PowerPoint (PPT)
- PDF to HTML
- PDF to RTF
- PDF to CSV
- Extract PDF to JSON
- Extract PDF to Markdown

Notice

- Table Recognition runs only when layout analysis is active (i.e. `enable_ai_layout = true`, or implicitly when `enable_ocr = true`).
- You need to load the DocumentAI model before using Table Recognition, **or** plug in your own table model via the callbacks described in 3.11 Use Custom AI Models via Callbacks.
- Setting `enable_ai_table_recognition = false` disables the table model entirely; detected table regions will then fall back to geometric reconstruction from the underlying page objects.
- The number of Table Recognition model instances can be tuned via the second parameter of `LibraryManager::SetDocumentAIModelCount`. See 3.2 Set DocumentAI Model.

Sample

This sample demonstrates how to convert a PDF to a DOCX file with Table Recognition enabled.

```
// Set the DocumentAI model path.
LibraryManager::SetDocumentAIModel("path/documentai.model");

ConvertOptions opt;
// Layout analysis must be enabled for Table Recognition to take effect.
opt.enable_ai_layout = true;
// Enable AI table recognition (enabled by default; set to false to disable).
opt.enable_ai_table_recognition = true;
CPDFConversion::StartPDFToWord("word.pdf", "password", "path/output.docx", opt);
```

3.11 Use Custom AI Models via Callbacks

Overview

Starting with **SDK v4.1.0**, ComPDF Conversion SDK exposes a callback-based extension point that lets you plug in your own AI inference engine for OCR, Layout Analysis, and Table Recognition. Instead of relying on the built-in DocumentAI model loaded by `LibraryManager::SetDocumentAIModel`, you can:

- Run inference with any model or runtime you choose (e.g. your in-house engine, PaddleOCR, a cloud OCR API).
- Return the result to the SDK as a JSON string with a well-defined schema.

When the relevant callback pair is registered on `CConvertCallback`, the SDK skips its built-in model invocation for that capability and consumes your JSON output instead. If a pair is left as `nullptr`, the SDK falls back to the built-in DocumentAI model (when available).

Callback Pairs

Each AI capability uses **two callbacks**: a *trigger* (invoked by the SDK with the path to a page image saved as PNG in a temp directory) and a *result getter* (invoked by the SDK immediately afterwards to retrieve the JSON string).

Capability	Trigger callback	Result getter callback	Triggered when
OCR	`ocr`	`get_ocr_result`	`enable_ocr = true`
Layout Analysis	`layout`	`get_layout_result`	`enable_ai_layout = true` or `enable_ocr = true`
Table Recognition	`table`	`get_table_result`	`enable_ai_table_recognition = true` **and** a table region is detected by layout analysis

Rules:

- The trigger receives a UTF-8 path to a PNG file. Return `true` if your inference succeeded, `false` to make the SDK ignore the result for that page.
- The getter must return a UTF-8 JSON C-string. The pointer must remain valid until the SDK has finished parsing it (typically until the next call to the same getter).
- Both callbacks for a capability must be set together. If only one is provided, the SDK falls back to the built-in path.
- Coordinates in your JSON must be in the **pixel space of the image the trigger received** (top-left origin, X right, Y down).
- Confidence filtering: OCR spans with `confidence < 0.1` and layout objects with `confidence < 0.45` are discarded by the SDK.

Sample

```
#include "compdf_conversion.h"
#include <string>

using namespace compdf;
using namespace compdf::common;
using namespace compdf::conversion;

// Buffers owned by the integrator. Must outlive each SDK getter call.
static std::string g_ocr_json;
```

```cpp
static std::string g_layout_json;
static std::string g_table_json;

// --- OCR ---
bool my_ocr_trigger(const char* image_path) {
    g_ocr_json = run_my_ocr_model(image_path);   // produce JSON (see schema below)
    return !g_ocr_json.empty();
}
const char* my_ocr_getter() { return g_ocr_json.c_str(); }

// --- Layout Analysis ---
bool my_layout_trigger(const char* image_path) {
    g_layout_json = run_my_layout_model(image_path);
    return !g_layout_json.empty();
}
const char* my_layout_getter() { return g_layout_json.c_str(); }

// --- Table Recognition ---
bool my_table_trigger(const char* image_path) {
    g_table_json = run_my_table_model(image_path);
    return !g_table_json.empty();
}
const char* my_table_getter() { return g_table_json.c_str(); }

int main() {
    LibraryManager::LicenseVerify("<license>", "<device_id>", "<app_id>");
    LibraryManager::Initialize("resource");
     /* SetDocumentAIModel is not required when callbacks cover OCR,
       Layout Analysis, and Table Recognition. */

    CConvertCallback callback   = {};
    callback.handle             = nullptr;
    callback.progress           = nullptr;
    callback.cancel             = nullptr;
    callback.ocr                = &my_ocr_trigger;
    callback.get_ocr_result     = &my_ocr_getter;
    callback.layout             = &my_layout_trigger;
    callback.get_layout_result  = &my_layout_getter;
    callback.table              = &my_table_trigger;
    callback.get_table_result   = &my_table_getter;

    ConvertOptions opt;
    opt.enable_ocr       = true;
    opt.enable_ai_layout = true;
    opt.languages        = { OCRLanguage::e_English };

    CPDFConversion::StartPDFToWord("input.pdf", "", "output.docx", opt, &callback);
    return 0;
}
```

You can register only the capabilities you want to override and leave the others as `nullptr` to keep the built-in behaviour.

Thread Safety and Lifetime

- Callbacks are invoked from the SDK conversion thread. Implement them in a thread-safe manner if your model engine is shared across calls.
- The PNG image at `image_path` lives in the SDK temp directory and may be deleted shortly after the trigger returns; copy or process it before returning.
- The JSON pointer returned from a getter must remain valid until the same getter is called again (or until the conversion task ends).

OCR Result JSON Schema

Returned by `get_ocr_result`. The SDK populates each `text_spans[].chars[]` either from `words[]` if provided, or by uniformly splitting the span rect.

```
{
  "text_spans": [
    {
      "text": "Hello World",
      "confidence": 0.98,
      "rotation": 0.0,
      "rect":  { "left": 120, "top": 80, "right": 320, "bottom": 110 },
      "style": {
        "font_size":  18.0,
        "font_color": { "r": 0, "g": 0, "b": 0 }
      },
      "words": [
        { "text": "Hello", "rect": { "left": 120, "top": 80, "right": 200, "bottom": 110 }
},
        { "text": "World", "rect": { "left": 210, "top": 80, "right": 320, "bottom": 110 } }
      ]
    }
  ]
}
```

Field	Type	Required	Description
`text_spans`	array	Yes	Recognized text spans on the page.
`text`	string	Yes	UTF-8 text content of the span.
`confidence`	number	No	0.0 – 1.0. Spans below 0.1 are discarded.
`rotation`	number	No	Text rotation in degrees. Default 0.
`rect`	object	Yes	Bounding box in image pixels (`left` / `top` / `right` / `bottom`).
`style.font_size`	number	No	Estimated font size in pixels.
`style.font_color`	object	No	`{ r, g, b }` 0 – 255.

Field	Type	Required	Description
`words`	array	No	Per-word boxes. If omitted, the SDK splits the span rect evenly. Strongly recommended for CJK + Latin mixed lines for correct glyph spacing.

Layout Analysis Result JSON Schema

Returned by `get_layout_result`. Objects with `confidence < 0.45` are discarded.

```
{
  "objects": [
    { "type": "title",     "confidence": 0.95, "rect": { "left": 60, "top": 50,  "right": 540, "bottom": 90  } },
    { "type": "paragraph", "confidence": 0.97, "rect": { "left": 60, "top": 100, "right": 540, "bottom": 220 } },
    { "type": "figure",    "confidence": 0.92, "rect": { "left": 80, "top": 240, "right": 520, "bottom": 460 } },
    { "type": "table",     "confidence": 0.93, "rect": { "left": 60, "top": 480, "right": 540, "bottom": 700 } }
  ]
}
```

Supported `type` values:

Value	Meaning
`paragraph`	Body text paragraph
`title`	Heading
`figure`	Image or figure
`figure_title`	Figure caption header
`figure_caption`	Figure caption text
`table`	Table region. Whether the table is bordered or borderless is determined by the table recognition stage, not by the layout label.
`table_title`	Table caption header
`table_caption`	Table caption text
`ordered_list`	Ordered list
`unordered_list`	Unordered list
`catalogue`	Table of contents
`formula`	Math formula
`code`	Code block

Value	Meaning
`algorithm`	Algorithm block
`header`	Page header
`footer`	Page footer
`page_number`	Page number
`reference`	Reference or citation

Objects with a `type` value that is not listed above are ignored. Use the values in this table as the canonical layout labels in your custom output.

Table Recognition Result JSON Schema

Returned by `get_table_result` once per detected table region. Polygons use 8 integers `[x0, y0, x1, y1, x2, y2, x3, y3]` in the order top-left, top-right, bottom-right, bottom-left.

```
{
  "type": "table_with_line",
  "position": [60, 480, 540, 480, 540, 700, 60, 700],
  "rows": 3,
  "cols": 2,
  "angle": 0.0,
  "height_of_rows": [40, 60, 60],
  "width_of_cols":  [200, 280],
  "table_cells": [
    {
      "start_row": 0, "end_row": 0,
      "start_col": 0, "end_col": 0,
      "cell_background_color_r": 240,
      "cell_background_color_g": 240,
      "cell_background_color_b": 240,
      "position": [60, 480, 260, 480, 260, 520, 60, 520]
    }
  ]
}
```

Field	Type	Description
`type`	string	`table_with_line` for bordered tables; any other value is treated as a non-standard (borderless) table.
`position`	int[8]	Table polygon in image pixels.
`rows` / `cols`	int	Row / column counts.
`angle`	number	Skew angle in degrees.
`height_of_rows`	int[]	Per-row pixel heights (length = `rows`).

Field	Type	Description
`width_of_cols`	int[]	Per-column pixel widths (length = `cols`).
`table_cells[]`	array	One entry per merged cell.
`start_row` / `end_row`	int	Inclusive row span of the cell.
`start_col` / `end_col`	int	Inclusive column span of the cell.
`cell_background_color_*`	int	Cell background color components (0 – 255).
`position`	int[8]	Cell polygon in image pixels.

Tip: Validate Your JSON

If you need a reference output to compare against, run a conversion once with the built-in DocumentAI model — the SDK uses the same JSON shape internally, so your custom output should follow the same structure.

3.12 Output Font Option

Overview

In some output formats, you can set the preferred font name to unify the default font style in the output document.

Supported Formats

The `font_name` option currently applies to the following formats:

- PDF to Word
- PDF to Excel
- PDF to PowerPoint
- PDF to Searchable PDF
- PDF to OFD

For Searchable PDF and OFD, `font_name` controls the font used for the invisible (or visible) text layer that is overlaid on the page background. For Word, Excel and PowerPoint, it sets the preferred default font of the generated document.

Example

The following example demonstrates how to set the preferred font name for the output document.

```
ConvertOptions opt;
opt.font_name = "Arial";
CPDFConversion::StartPDFToWord("word.pdf", "password", "path/output.docx", opt);
```

3.13 Convert PDF to Word

Overview

Converting PDF to Word is an operation that converts the PDF format file into a editing Word format file. By converting PDF to Word, you can easily edit, modify, insert, or delete text and pictures, adjust layout and properties.

Layout differences

- **Word's Streaming Layout:** Ideal for editing, with your editing, the content dynamically adapts to different positions. However, a Word file would display differently due to the incompatibility of various software or app versions. It makes it unsuitable for precise documentation like electronic files or certificates.
- **PDF's Fixed Page Layout:** Ensures a stable, uniform appearance and print quality across all devices. The content and formatting are locked upon creation, making alterations difficult without affecting the overall layout. It's preferred for formal documentation such as business reports and official electronic records.

Sample

This sample demonstrates how to convert from a PDF to DOCX file.

```
ConvertOptions opt;
// Set Reserved page layout.
opt.page_layout_mode = PageLayoutMode::e_Box;
CPDFConversion::StartPDFToWord("word.pdf", "password", "path/output.docx", opt);

// Set Streaming layout.
opt.page_layout_mode = PageLayoutMode::e_Flow;
CPDFConversion::StartPDFToWord("word.pdf", "password", "path/output.docx", opt);
```

Convert Formulas to Images

When a document contains complex formulas and you want to preserve visual consistency in the output document, you can enable the `formula_to_image` option.

```
ConvertOptions opt;
opt.formula_to_image = true;
CPDFConversion::StartPDFToWord("word.pdf", "password", "path/output.docx", opt);
```

3.14 Convert PDF to Excel

Overview

ComPDF Conversion SDK supports converting PDF documents to Microsoft Excel format (.xlsx). By extracting, parsing, and importing data from PDF into Excel, users can further edit, analyze, or share Excel files. This feature helps increase productivity, reduce manual entry errors, and simplify complex document processing tasks.

Set the content options for Excel

When converting PDF files to Excel files, you need to pay attention to the settings of the following options, which will directly affect the content written to the Excel file.

- Content options:
  If you set the `excel_all_content` option, The converted Xlsx file will contain all the contents in the PDF.
- Worksheet options:

Option	Description
`ExcelWorksheetOption::e_ForTable`	Create one sheet for one table.
`ExcelWorksheetOption::e_ForPage`	Create one sheet for one PDF page.
`ExcelWorksheetOption::e_ForDocument`	Create one sheet for the entire PDF document.

Sample

This sample demonstrates how to convert from a PDF to XLSX file.

```
ConvertOptions opt;
CPDFConversion::StartPDFToExcel("excel.pdf", "password", "path/output.xlsx", opt);

// Set all content options.
opt.excel_all_content = true;
opt.excel_worksheet_option = ExcelWorksheetOption::e_ForDocument;
CPDFConversion::StartPDFToExcel("excel.pdf", "password", "path/output.xlsx", opt);
```

3.15 Convert PDF to PowerPoint

Overview

ComPDF Conversion SDK provides the function of converting PDF files to PowerPoint files and restoring the layout and format of the original document, which can meet the needs of users for the presentation and editing of document content in Microsoft PowerPoint.

Sample

This sample demonstrates how to convert from a PDF to PPTX file.

```
ConvertOptions opt;
// PDF to PPT.
CPDFConversion::StartPDFToPpt("ppt.pdf", "password", "path/output.pptx", opt);
```

3.16 Convert PDF to HTML

Overview

ComPDF Conversion SDK provides the PDF to HTML function, which can convert PDF files to HTML files while maintaining the layout and format of the original document, allowing users to browse and view the document on Web.

Notice

When converting PDF to HTML format, ComPDF Conversion SDK provides the following four options to create HTML files:

Options	Description
`HtmlOption::e_SinglePage`	Convert the entire PDF file into a single HTML file, where all PDF pages are connected in sequence according to page number, displayed on the same HTML page.
`HtmlOption::e_SinglePageWithBookmark`	Convert the PDF file into a single HTML file with an outline for navigation at the beginning of the HTML page. Still, all PDF pages are connected in sequence according to page number, displayed on the same HTML page.
`HtmlOption::e_MultiPage`	Convert the PDF file into multiple HTML files. Each HTML file corresponds to a PDF page, and users can navigate to the next HTML file via a link at the bottom of the HTML page.
`HtmlOption::e_MultiPageWithBookmark`	Convert the PDF file into multiple HTML files. Each HTML file corresponds to a PDF page, and users can navigate to the next HTML file via a link at the bottom of the HTML page. The links of all the HTML files are presented in an outline HTML file for navigation.

Sample

This sample demonstrates how to convert from a PDF to HTML file.

```
ConvertOptions opt;
// Convert using default options.
CPDFConversion::StartPDFToHtml("html.pdf", "password", "path/output.html", opt);

// Set the html page display method to HtmlOption::e_MultiPageWithBookmark.
opt.page_layout_mode = PageLayoutMode::e_Box;
opt.html_option = HtmlOption::e_MultiPageWithBookmark;
CPDFConversion::StartPDFToHtml("html.pdf", "password", "path/output.html", opt);
```

3.17 Convert PDF to CSV

Overview

ComPDF Conversion SDK supports converting PDF documents to CSV (Comma-Separated Values). Converting PDF to CSV is a common need, usually used to extract tabular or structured data from PDF documents and convert them into CSV files.

Set Whether to Automatically Create Folders

When multiple CSV files may be output, you can control whether to automatically create folders to store the CSV files by setting the `auto_create_folder` option. When this option is enabled, a folder with the same name as the output file will be automatically created in the output path to store the CSV files.

Sample

This sample demonstrates how to convert from a PDF to CSV file.

```
ConvertOptions opt;
opt.excel_csv_format = true;
CPDFConversion::StartPDFToExcel("csv.pdf", "password", "path/output.csv", opt);

// Merge all tables into one CSV file
opt.excel_worksheet_option = ExcelWorksheetOption::e_ForDocument;
CPDFConversion::StartPDFToExcel("csv.pdf", "password", "path/output.csv", opt);
```

3.18 Convert PDF to Image

Overview

ComPDF Conversion SDK provides an API for converting PDF to images. Integrate ComPDF Conversion SDK to your apps to convert PDF into images easily.

Setting Image Formats

In ComPDF Conversion SDK, supported image formats include:

- JPG
- JPEG
- JPEG2000
- PNG
- BMP
- TIFF
- TGA
- GIF
- WEBP

Setting Image Color Modes

Supported image color modes in ComPDF Conversion SDK include:

- **Color:** Color mode, where the image effect is consistent with the original PDF page.
- **Gray:** Grayscale mode.
- **Binary:** Black and white mode, which applies binarization to the original effect.

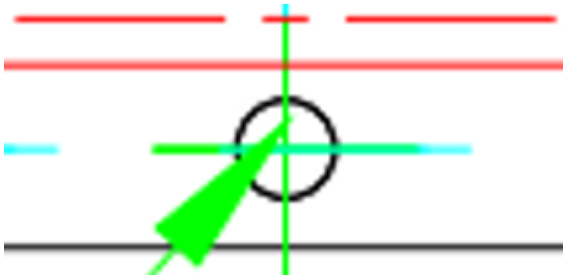
Setting Image Scaling

The SDK supports setting image scaling. The default scaling is 1.0, which maintains the original PDF page size. If you want to double the image size, you can set `image_scaling` to 2.0; similarly, to reduce the image size by half, set `image_scaling` to 0.5.

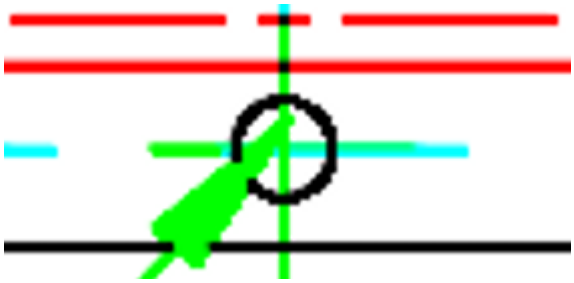
Enhancing Image Path Display

The SDK supports an option called `image_path_enhance` for enhancing the display of image paths. This option can be enabled when you want to enhance the display effect of paths within the PDF page.

- Not enable `image_path_enhance` option(original PDF rendering effect):



- Enable `image_path_enhance` option:



Notice

- A higher `image_scaling` value results in images with higher resolution, but it also increases memory usage and slows down the conversion.
- A higher `image_scaling` value does not necessarily equate to higher clarity; the clarity also depends on the original image resolution in the document.

Sample

The following complete example code demonstrates how to convert a PDF document into PNG format.

```
ConvertOptions opt;
// Convert PDF to Image (JPEG).
opt.image_type = ImageType::e_JPEG;
CPDFConversion::StartPDFToImage("jpeg.pdf", "password", "path/output", opt);

// Convert PDF to Image (PNG) and set image_scaling to double the original PDF size.
opt.image_type = ImageType::e_PNG;
opt.image_scaling = 2.0;
CPDFConversion::StartPDFToImage("png.pdf", "password", "path/output", opt);
```

3.19 Convert PDF to RTF

Overview

RTF is a popular text format that can retain the format and style data of the text, and it is convenient for most text readers to read and write. Integrate ComPDF Conversion SDK to convert PDF to RTF files now.

Sample

This sample demonstrates how to convert from a PDF to RTF file.

```
ConvertOptions opt;
// PDF to RTF.
CPDFConversion::StartPDFToRtf("rtf.pdf", "password", "path/output.rtf", opt);
```

3.20 Convert PDF to TXT

Overview

When you need to extract the text content in the PDF file, in order for data analysis, text mining, information retrieval, etc. Using ComPDF Convert SDK, you can easily extract the text in the PDF into the TXT file.

Preserving Table Format

The SDK supports an option called `txt_table_format` that preserves the table format when writing the TXT file, meaning that the original table structure is maintained. It is generally recommended to enable this option, especially useful for data extraction scenarios.

Sample

This sample demonstrates how to convert from a PDF to TXT file.

```
ConvertOptions opt;
// PDF to TXT.
CPDFConversion::StartPDFToTxt("txt.pdf", "password", "path/output.txt", opt);
```

3.21 Convert PDF to Searchable PDF

Overview

To make a searchable PDF by adding invisible text to an image based PDF such as a scanned document using OCR.

Set Transparent Text Layer

When outputting a searchable PDF, you can use the following option to control the hidden text layer:

- `transparent_text`: Whether to output a transparent text layer.

Sample

The following example demonstrates how to convert a PDF document to a searchable PDF file.

```
LibraryManager::SetDocumentAIModel("path/documentai.model");

ConvertOptions opt;
opt.enable_ocr = true;
opt.languages = {OCRLanguage::e_English};
opt.transparent_text = true;
CPDFConversion::StartPDFToSearchablePdf("scan.pdf", "password", "path/output.pdf", opt);
```

3.22 Convert PDF to OFD

Overview

ComPDF Conversion SDK supports converting PDF documents to OFD documents. Similar to searchable PDF, OFD conversion also supports OCR, page background preservation, and transparent text layers.

Notice

- If you need to generate searchable OFD output, you can enable `transparent_text`.
- When `enable_ocr` is enabled, it is recommended to specify the OCR language through `languages`.

Sample

```
LibraryManager::SetDocumentAIModel("path/documentai.model");

ConvertOptions opt;
opt.enable_ocr = true;
opt.languages = {OCRLanguage::e_English};
opt.contain_page_background_image = true;
opt.transparent_text = true;
CPDFConversion::StartPDFToOfd("scan.pdf", "password", "path/output.ofd", opt);
```

3.22 Releasing Library Resources

Overview

Releases the files and memory resources occupied by the ComPDF Conversion SDK.

Notice

- After calling this interface to release library resources, the ComPDF Conversion SDK will no longer function properly and must be reloaded.
- When you only want to release the resources occupied by the AI model rather than all resources used by the ComPDF Conversion SDK, you can achieve this by calling the `ReleaseDocumentAIModel` interface.

Sample

```
// Release AI Model resources.
LibraryManager::ReleaseDocumentAIModel();

// Release library resources.
LibraryManager::Release();
```

4 Data Extraction Guide

Unleash the Power of Data with ComPDF Conversion SDK's Data Extraction to detect, recognize, analyze, and extract the PDF text, image, table, etc.

4.1 Extract PDF To JSON

Overview

Extract text, tables and images from PDF documents to Json file.

Standard table and non-standard table

Commonly, tables can be divided into two categories: standard tables and non-standard tables. The specific definitions are as follows:

- Standard table: The table border and the inner lines of the table are complete and clear. There is no

need to manually add table lines to divide the table content.

EXTERIOR - RAMPS, DOORS, and LIFTS			
Spec #	Spec	Initials	Notes
101	**REMOVAL OF ITEMS FROM WORK AREA** Homeowner is responsible for removing objects from the work area. However, contractor is to double check that the items have been removed and that the area is prepared for conversion.		
102	**INITIAL INSPECTION AND REPAIR** Inspect the general area (concrete, doorway, threshold, etc.) for damage. Immediately report any "unforeseen items" to the Care Manager and Koremen.		
103	**PREPARE YARD** Prepare the existing yard for the ramp modification and cement. This includes the addition of dirt as needed, the excavating of the land, leveling, and grading of soil away from the home.		
104	**DEMOLITION** Demolish and remove existing areas *only as necessary* to construct the modification outlined in the specifications. Dispose of tear out in a code legal dump. Follow all environmental protection measures (lead safe practices) as required by the EPA and local ordinances.		Existing landing and ramps
105A	**DOOR THRESHOLD LANDING FOR VPL** Construct a preservative treated wood landing at the door threshold height, approximately 42" above ground level or the concrete. 5'x6'. Add new steps. The landing should attach to existing porch and connect to new VPL. (Refer to diagram)		McKinley Collins approved aluminum modular platform and stairs with hand rails.
106	**VERTICAL PLATFORM LIFT** Install a Bruno (or equivalent) vertical platform lift system per manufacturer's specifications. Ensure the lift has access to appropriate electrical outlets. Lift to have battery backup and call capability from bottom or top.		Pour a concrete base for the VPL. Ensure level. Must have manual override for power outages. Ensure smooth operation.
106a	**ELECTRIC FOR INSTALLING VPL** Check with local zoning, fire, electric, and other code enforcement agencies regarding contractor licensing required for installation. Install based upon these requirements.		Add outlet to side of house for VPL.
115	**CONCRETE LANDING** Excavate area to create new concrete landing and loading area. Form and pour 4" thick, 25SF, 2200 PSI concrete slab, to create an area to allow VPL stationary installation and client access point. Use straight, solid forms between temps of 40-100F. All concrete shall be: o free of voids and cavities. o treated with liquid curing compound. o protected from the weather while curing. o broom finished across direction of traffic. Create a smooth transition onto the driveway/sidewalk from landing. Remove all forms, re-grade and spot seed.		Extend the concrete a minimum 5' wide to the edge of the home.

- **Non-Standard Tables:** Tables lacking borders or clear inner lines, requiring manual additions of table lines to separate contents.

features	precision	recall	F1-score	MCC	AUROC
B	0.823 ± 0.311	0.024 ± 0.012	0.047 ± 0.022	$\mathbf{0.130} \pm 0.055$	0.799 ± 0.013
BC	0.818 ± 0.173	0.057 ± 0.016	0.106 ± 0.028	$\mathbf{0.197} \pm 0.045$	0.842 ± 0.005
BT	0.542 ± 0.177	0.051 ± 0.030	0.092 ± 0.052	$\mathbf{0.138} \pm 0.060$	0.786 ± 0.009
BV	0.745 ± 0.040	0.232 ± 0.047	0.350 ± 0.068	$\mathbf{0.372} \pm 0.055$	0.815 ± 0.006
BY	0.781 ± 0.022	0.153 ± 0.014	0.256 ± 0.020	$\mathbf{0.314} \pm 0.016$	0.820 ± 0.004
BTV	0.709 ± 0.011	0.268 ± 0.013	0.389 ± 0.015	$\mathbf{0.393} \pm 0.013$	0.832 ± 0.003
BCTVY	$\mathbf{0.793} \pm 0.009$	$\mathbf{0.424} \pm 0.011$	$\mathbf{0.552} \pm 0.010$	$\mathbf{0.541} \pm 0.008$	$\mathbf{0.890} \pm 0.002$

Table Extraction Option

ComPDF Conversion SDK supports the option `json_contain_table`, when enabled, will extract table content from PDFs and output the table structure; otherwise, table content will be treated as regular text.

Notice

- Without enabling AI layout analysis or OCR options, tables in the original PDF cannot be extracted. It is recommended to enable AI layout analysis or OCR for high-precision table recognition.

Sample

Full sample code which illustrates the text extraction capabilities.

```
ConvertOptions opt;
// Extract PDF to JSON.
CPDFConversion::StartPDFToJson("json.pdf", "password", "path/output.json", opt);
```

4.2 Extract PDF To Markdown

Overview

Extract text, tables and images from PDF documents to Markdown file.

Sample

Full code sample which shows how to convert from a PDF to Markdown file.

```
ConvertOptions opt;
// Extract PDF to Markdown.
CPDFConversion::StartPDFToMarkdown("markdown.pdf", "password", "path/output.md", opt);
```

5. Support

5.1 FAQ

- Does OCR work on x86 architecture?

  Currently, the OCR only works on x64 architecture.

5.2 Contact Us

Thanks for your interest in ComPDF Conversion SDK, the easy-to-use and powerful development solution. If you encounter technical questions or bug issues when using ComPDF Conversion SDK, please submit the problem report to the ComPDF team. More information as follows would help us to solve your problem:

- ComPDF Conversion SDK product and version.
- Your operating system and IDE version.
- Detailed descriptions of the problem.

- Any other related information, such as an error screenshot.

Contact Inforation

- Home link: https://www.compdf.com
- Technical Support: https://www.compdf.com/support
- Email: support@compdf.com

Thanks,
The ComPDF Team